

Bayesian Physics-Informed Extreme Learning Machine for Forward and Inverse PDE Problems with Noisy Data

Xu Liu, Wen Yao, Wei Peng, and Weien Zhou

1 Introduction

The Physics-Informed Extreme Learning Machine (PIELM) has gained attention as a rapid version of Physics-Informed Neural Networks (PINNs) for solving partial differential equations (PDEs). PIELM fixes the input layer weights with random values and uses Moore-Penrose generalized inverse to calculate the output weights, resulting in significantly faster computation compared to traditional PINNs. However, PIELM has two major limitations when dealing with noisy data:

1. It easily suffers from overfitting noisy data
2. The accuracy is drastically sensitive to the number of hidden neurons under noisy conditions
3. It lacks built-in uncertainty quantification capabilities

This paper introduces a novel Bayesian Physics-Informed Extreme Learning Machine (BPIELM) framework to address these limitations, enabling both forward and inverse PDE problems to be solved efficiently with noisy data while providing uncertainty quantification.

2 Problem Setup

The paper considers linear PDEs of the form:

$$\mathcal{L}u = f_k(x, y), \quad (x, y) \in \Omega, \quad (1)$$

$$\mathcal{B}[u(x, y)] = b(x, y), \quad (x, y) \in \partial\Omega, \quad (2)$$

where $\mathcal{L}$ is a linear differential operator, $\mathcal{B}$ is a boundary/initial condition operator, and k is a problem parameter.

Two types of problems are considered:

1. **Forward PDE problems:** $\mathcal{L}$, $\mathcal{B}$, and k are known, while $u(x, y)$ is unknown. Given noisy boundary data $\hat{b}$, the goal is to determine the solution $u(x, y)$ and quantify its uncertainty.
2. **Inverse PDE problems:** $\mathcal{L}$ and $\mathcal{B}$ are known, but the problem parameter k is unknown, and $u(x, y)$ is partially known. Given noisy data $\hat{u}$ and $\hat{b}$, the goal is to infer both the solution $u(x, y)$ and problem parameter k , and quantify their uncertainties.

In both cases, measurements are assumed to follow Gaussian distributions with known standard deviations centered at the true values:

$$\hat{b}_i = b(x_b^{(i)}, y_b^{(i)}) + \epsilon_b^{(i)}, \quad i = 1, 2, \dots, N_b \quad (3)$$

$$\hat{u}_i = u(x_u^{(i)}, y_u^{(i)}) + \epsilon_u^{(i)}, \quad i = 1, 2, \dots, N_u \quad (4)$$

where $\epsilon_b^{(i)}$ and $\epsilon_u^{(i)}$ are independent Gaussian noises with zero mean and known standard deviations $\sigma_b^{(i)}$ and $\sigma_u^{(i)}$.

3 Methodology

3.1 Review of Related Methods

3.1.1 Physics-Informed Neural Networks (PINNs)

PINNs encode physical laws into the loss function to constrain the space of admissible solutions:

$$\mathcal{L}(h) = w_f \mathcal{L}_f(h, T_f) + w_b \mathcal{L}_b(h, T_b) + w_u \mathcal{L}_u(h, T_u) \quad (5)$$

where $\mathcal{L}_f$, $\mathcal{L}_b$, and $\mathcal{L}_u$ are losses for PDE residuals, boundary constraints, and data constraints, respectively. PINNs use gradient-based optimization to find the best parameters h .

3.1.2 Physics-Informed Extreme Learning Machine (PIELM)

PIELM uses a single-layer feedforward neural network with fixed random input weights and trains only the output weights. The linear nature of PDEs combined with physical laws leads to a linear equation:

$$Hc = Y \quad (6)$$

where c represents the unknown output weights. The solution is obtained using Moore-Penrose generalized inverse: $c_{pred} = H^\dagger Y$.

3.2 Bayesian Physics-Informed Extreme Learning Machine (BPIELM)

BPIELM combines PIELM with Bayesian methods to address the limitations of PIELM. It consists of two main components:

1. **Prior for Extreme Learning Machine with Physical Laws:** BPIELM incorporates physical laws into ELM and introduces a prior distribution on output weights
2. **Bayesian Method for Estimating Posterior Distributions:** BPIELM uses Bayesian inference to estimate posterior distributions of parameters and quantify uncertainty

3.2.1 The BPIELM Algorithm

Algorithm 1 Bayesian Physics-Informed Extreme Learning Machine

- 1: **Input:**
 - 2: $\hat{b}$: noisy measurements on the boundary $\partial\Omega$
 - 3: $\hat{u}$: noisy measurements in the domain Ω (for inverse problems)
 - 4: **Output:** $u(x, y)$ and uncertainty (also k for inverse problems)
 - 5: // The prior for extreme learning machine with physical laws
 - 6: Construct a single-layer NN with N neurons and initialize input layer weights randomly
 - 7: Define training errors n_f , n_b (and n_u for inverse problems)
 - 8: Transform the linear system into a linear equation $Hx = Y$
 - 9: Suppose the prior distribution of parameters $p(x|g)$ follows a Gaussian distribution with hyper-parameter g
 - 10: // The Bayesian method for estimating posterior distributions
 - 11: Define the likelihood function $p(Y|x, \sigma^2)$ with hyper-parameter σ
 - 12: Construct the posterior by the prior distribution and the likelihood function: $p(x|Y, g, \sigma^2) = \frac{p(Y|x, \sigma^2)p(x|g)}{p(Y|g, \sigma^2)}$
 - 13: Use Evidence Procedure to optimize hyper-parameters g and σ
 - 14: Calculate output Y_{new} by the integral of the posterior distribution for input instances v_{new} and obtain k by the new output weights for inverse problems
 - 15: Obtain the uncertainty by the variance: $\sigma^2(v_{new}) = \sigma^2 + v_{new}^T R v_{new}^T$, where $R = (gI + \sigma^{-2} H^T H)^{-1}$
-

3.2.2 Mathematical Details

BPIELM uses a single-layer neural network with N neurons. The parameters of the input layer are defined as $v = [x, y, 1]^T$, $a = [a_1, a_2, \dots, a_n]^T$, $b = [b_1, b_2, \dots, b_n]^T$, $c = [c_1, c_2, \dots, c_n]^T$ (input weights), with activation function $\phi = \tanh$ and output weights $x = [x_1, x_2, \dots, x_n]^T$.

The output of the neural network is:

$$\hat{u}(v) = \phi(z)x \tag{7}$$

where $z_k = [a_k, b_k, c_k] \cdot v$.

The partial derivatives needed for PDE operators can be expressed as:

$$\frac{\partial^k \hat{u}}{\partial x^k} = \sum_{j=1}^n x_j a_j^k \frac{\partial^k \phi}{\partial z^k} \quad (8)$$

$$\frac{\partial^k \hat{u}}{\partial y^k} = \sum_{j=1}^n x_j b_j^k \frac{\partial^k \phi}{\partial z^k} \quad (9)$$

Physical laws are incorporated by introducing physics-based training errors:

$$n_f = \mathcal{L}[\phi(z)x] - f_k(x, y) = 0 \quad (10)$$

$$n_b = \mathcal{B}[\phi(z)x] - \hat{b}(x, y) = 0 \quad (11)$$

For forward problems, this leads to a linear system $Hx = Y$ where $H \in \mathbb{R}^{N_c \times N}$, $N_c = N_f + N_b$, and N is the neuron number.

For inverse problems, the PDE error can often be written by separating variables:

$$n_f = \mathcal{L}[\phi(z)x] + \sum_{j=1}^m \phi_j(z)k_j - \tilde{f}(x, y) = 0 \quad (12)$$

This leads to a linear system:

$$\tilde{H}\tilde{x} = \tilde{Y} \quad (13)$$

where $\tilde{x} = [x, k]^T$, treating problem parameters as additional output weights.

To apply Bayesian methods, a prior distribution is introduced for output weights:

$$p(x|g) = \mathcal{N}(0, g^{-1}I) \quad (14)$$

Assuming measurements follow a Gaussian distribution, the likelihood function is:

$$p(Y|x, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{N_c/2}} \exp\left(-\frac{\|Y - Hx\|^2}{\sigma^2}\right) \quad (15)$$

The posterior distribution then follows:

$$p(x|Y, g, \sigma^2) = \frac{p(Y|x, \sigma^2)p(x|g)}{p(Y|g, \sigma^2)} \quad (16)$$

This posterior is a Gaussian distribution with mean μ and variance R :

$$\mu = \sigma^{-2} R H^T Y \quad (17)$$

$$R = (gI + \sigma^{-2} H^T H)^{-1} \quad (18)$$

The hyper-parameters g and σ are optimized using the Evidence Procedure:

$$g \approx \frac{N - g \cdot \text{trace}[R]}{[\mu]^T R^T \mu} \quad (19)$$

$$\sigma^2 \approx \frac{\|Y - Hx\|^2}{N_c - N + g \cdot \text{trace}[R]} \quad (20)$$

For a new input v_{new} , the output follows the distribution:

$$p(Y_{new}|Y, g, \sigma^2) = \mathcal{N}(H_{new}x, \sigma^2(v_{new})) \quad (21)$$

where the predictive variance is:

$$\sigma^2(v_{new}) = \sigma^2 + v_{new}^T R v_{new}^T \quad (22)$$

4 Numerical Results and Analysis

The paper examines BPIELM on several forward and inverse PDE problems:

4.1 Forward Problems

4.1.1 Poisson Equation

$$u_{xx} + u_{yy} = 16x^2 + 64y^2 - (2 - 4)e^{-(2x^2+4y^2)}, \quad (x, y) \in \Omega \quad (23)$$

with exact solution $u = \frac{1}{2} + e^{-(2x^2+4y^2)}$.

Key findings:

- BPIELM provides more accurate predictions than PIELM (MAEs of 0.019 vs. 0.029) with uncertainty quantification
- Areas with larger errors correspond to larger standard deviations in BPIELM
- BPIELM is more robust to increasing noise levels than PIELM
- BPIELM outperforms PIELM with fewer sensors

- BPIELM is less sensitive to neuron number than PIELM - PIELM performs worse when neuron number exceeds constraint count
- BPIELM is computationally much more efficient than PINN while achieving better MAE

4.1.2 Advection Equation

$$\frac{\partial u}{\partial t} - c \frac{\partial u}{\partial x} = 0, \quad (x, t) \in \Omega \quad (24)$$

with wave speed $c = -2$ and exact solution $u(x, t) = 2\text{sech}(3n - \frac{1}{2})$, where $n = \text{mod}(x - x_0 + ct + \frac{L}{2}, L)$.

Key findings:

- Under high noise conditions, BPIELM achieves MAE of 0.066 vs. PIELM's 0.070
- BPIELM errors are mostly bounded by two standard deviations
- BPIELM significantly outperforms PIELM with fewer sensors
- BPIELM shows less sensitivity to neuron number than PIELM
- BPIELM achieves better accuracy than both PIELM and PINN while being two orders of magnitude faster than PINN

4.1.3 Diffusion Equation

$$\frac{\partial u}{\partial t} - \nu \frac{\partial^2 u}{\partial x^2} = f(x, t), \quad (x, t) \in \Omega \quad (25)$$

with diffusion coefficient $\nu = 0.01$ and an oscillatory exact solution.

Key findings:

- BPIELM achieves MAE of 0.0186 vs. PIELM's 0.0787
- BPIELM performs especially well compared to PIELM under high noise conditions
- BPIELM maintains accuracy with fewer sensors
- BPIELM is less sensitive to neuron number than PIELM
- BPIELM achieves better accuracy than both PIELM and PINN while maintaining computational efficiency

4.2 Inverse Problems

4.2.1 Poisson Equation with Unknown Parameters

$$u_{xx} = -k_1 \sin(0.7x) - k_2 \cos(1.5x), \quad x \in \Omega \quad (26)$$

with unknown parameters k_1 and k_2 and exact solution $u = \sin(0.7x) + \cos(1.5x) - 0.1x$.

Key findings:

- BPIELM accurately identifies unknown parameters ($k_1 = 0.49$, $k_2 = 2.25$) while providing uncertainty estimates
- BPIELM (0.490, 2.300) outperforms PIELM (0.484, 2.390) for parameter estimation under noise
- BPIELM shows less sensitivity to neuron number than PIELM
- BPIELM is computationally more efficient than PINN while achieving comparable accuracy

4.2.2 Helmholtz Equation with Unknown Parameters

$$u_{xx} + \kappa^2 u = -k_1 f_1(x) + k_2 f_2(x) - k_3, \quad x \in \Omega \quad (27)$$

with wave number $\kappa^2 = 10$ and three unknown parameters k_1 , k_2 , and k_3 .

Key findings:

- BPIELM accurately identifies the three unknown parameters ($k_1 = 10$, $k_2 = 16$, $k_3 = -10$)
- BPIELM (9.80, 16.0, -10) outperforms PIELM (9.71, 14.8, -10) for parameter estimation
- BPIELM is dramatically less sensitive to neuron number than PIELM
- BPIELM achieves training times two orders of magnitude faster than PINN

5 Implications and Advantages

BPIELM offers several significant advantages:

1. **Built-in Uncertainty Quantification:** BPIELM provides uncertainty estimates for solutions and parameters, making it suitable for applications requiring reliability assessments.
2. **Improved Accuracy:** BPIELM consistently produces more accurate predictions than PIELM under noisy conditions by estimating probability distributions instead of directly fitting noisy data.
3. **Reduced Overfitting:** BPIELM avoids overfitting by using Bayesian methods to regularize the learning process, making it more robust to noise.
4. **Reduced Sensitivity to Neuron Number:** Unlike PIELM, which is highly sensitive to neuron count, BPIELM maintains good performance across a wider range of network configurations.
5. **Computational Efficiency:** BPIELM is significantly faster than PINN (orders of magnitude) while maintaining comparable or better accuracy.
6. **Unified Framework:** BPIELM handles both forward and inverse problems in a unified framework, treating problem parameters as additional output weights.

However, BPIELM inherits some limitations from PIELM, most notably the restriction to linear PDEs. Future work could focus on extending the method to nonlinear problems.